

Step 1 - Understand the Problem and Establish Design Scope

The first step in designing a **Search Autocomplete System** is to clarify the requirements with the interviewer.

Since system design questions are intentionally open-ended, asking clarification questions helps:

- Define the system scope
- Avoid incorrect assumptions
- Identify constraints early

Candidate – Interviewer Discussion

Candidate:

Is matching supported only at the beginning of a query or also in the middle?

Interviewer:

Only at the beginning of the search query.

Candidate:

How many autocomplete suggestions should be returned?

Interviewer:

5 suggestions.

Candidate:

How are the 5 suggestions selected?

Interviewer:

Suggestions are ranked based on popularity using historical search frequency.

Candidate:

Does the system support spell correction or autocorrect?

Interviewer:

No.

Candidate:

Are queries only in English?

Interviewer:

Yes. We may discuss multi-language support later if time permits.

Candidate:

Do queries contain uppercase letters or special characters?

Interviewer:

No. Queries only contain lowercase alphabetic characters.

Candidate:

What is the scale of the system?

Interviewer:

10 million daily active users (DAU).

Final Requirements Summary

After clarifying the requirements, the system should support the following characteristics.

1. Fast Response Time

Autocomplete suggestions must appear almost instantly while the user types.

Industry expectation:

None

< 100 milliseconds

If the response is slow:

- Typing becomes laggy
- UI stutters
- User experience degrades

Fast response time is one of the most critical requirements.

2. Relevant Suggestions

Suggestions must closely match the typed prefix.

Example:

If user types:

None

app

Expected suggestions:

- apple

- application
- app store

Irrelevant results reduce usability.

3. Sorted Results

Suggestions should be ranked based on:

- Historical search frequency
- Popularity
- Query trends

The most commonly searched queries should appear first.

Example:

None

1. apple

2. application

3. app store

4. Scalability

The system must support:

None

10 million DAU

This means:

- Massive query traffic
- Millions of concurrent requests
- Continuous query updates

The architecture should support horizontal scaling.

5. High Availability

The autocomplete system should remain operational even if:

- Some servers fail
- Network issues occur
- Databases become slow

Users expect search suggestions to always work.

Assumptions

The interview scope assumes:

Feature	Assumption
Matching	Prefix matching only
Language	English only
Characters	Lowercase alphabets only
Suggestions Count	Top 5
Ranking	Based on popularity

Spell Check	Not supported
-------------	---------------

Important System Characteristics

Requirement	Importance
Low Latency	Critical
Scalability	Critical
Availability	Critical
Ranking Accuracy	High
Fault Tolerance	High

Challenges Identified

From the requirements, we can identify several technical challenges:

Challenge	Description
-----------	-------------

Real-Time Speed	Must respond within milliseconds
Massive Traffic	Millions of search requests
Efficient Prefix Search	Fast lookup for typed prefixes
Ranking	Return top-k popular queries
Memory Efficiency	Huge search dataset
High Availability	No downtime

Expected Topics in Later Steps

The remaining design process will typically include:

1. Back-of-the-envelope estimation
2. High-level architecture
3. Trie-based data structure
4. Ranking algorithm
5. Caching strategy
6. Distributed system design
7. Data update pipeline
8. Fault tolerance and scaling